* Module 02 -- Hirely Project Explanation in Hinglish

CONFIDENTIAL & PRIVATE STUDY MATERIAL
Target Roles: AI Engineer, Applied AI Engineer, Backend AI Engineer
<- Back to Study Notes Index - Previous Module: Project Story <- - Next Module: Backend Architecture ->

[INDEX] Table of Contents

- 1. Ye Project Kya Hai? (What is Hirely?)
- 2. Ye Problem Kyun Solve Karta Hai? (Why was it built?)
- 3. User Is Project Mein Kya Karta Hai? (User Journey)
- 4. Backend Request Kaise Process Karta Hai? (Request Lifecycle)
- 5. AI Features Kaise Kaam Karte Hain? (AI Mechanics)
- 6. Database Ka Role Kya Hai? (PostgreSQL & SQLAlchemy)
- 7. RAG Kya Karta Hai Aur Kyun Zaroori Hai? (Candidate Evidence Assistant)
- 8. Docker Kyun Use Kiya? (Containerization Explained)
- 9. Project Mein Kya Limitations Hain? (Real Production Gaps)
- 10. Quick Revision Cheat-Sheet

1. Ye Project Kya Hai? (What is Hirely?)

Simple Samajho:

Hirely ek **AI-powered recruitment platform** hai jo do logon ko aapas mein jodta hai:

1. **Candidate (Job Seeker):** Jo achhi job dhoondh raha hai aur apna resume improve karna chahta hai.
2. **Recruiter (HR / Hiring Manager):** Jo open positions ke liye best candidates ko bina ghanton resume padhe efficiently shortlist karna chahta hai.

Technical Angle:

Ye koi simple "ChatGPT ka wrapper" nahi hai. Hirely ek full-stack system hai jo **Python (FastAPI)**, **PostgreSQL**, **Google Gemini AI**, **Dense Vector Embeddings**, aur **RAG (Retrieval-Augmented Generation)** par bana hai.

Sabse khaas baat ye hai ki Hirely mein **security, business rules, aur matching scores AI par nahi chhode gaye hain**. Wo sab deterministic Python backend code calculate karta hai. AI ka use sirf un cheezon ke liye hota hai jahan language understanding aur evidence extraction chahiye.

2. Ye Problem Kyun Solve Karta Hai? (Why was it built?)

* Candidate Ki Problem:

Socho tumne kisi job ke liye apply kiya. Tumhe nahi pata ki tumhara resume ATS (Applicant Tracking System) ko pass karega ya reject ho jayega. Koi feedback nahi milta, aur har job ke liye naya cover letter likhna bohot time-consuming hota hai.

* Recruiter Ki Problem:

Ek software engineer job opening aayi, aur 2 din mein 500 resumes aa gaye. HR ke paas har resume ko line-by-line padhne ka time nahi hota. Agar purana ATS use karein jo sirf exact keywords dhundta hai, to wo un candidates ko reject kar deta hai jinhone "Python FastAPI" ki jagah "Backend Web Development with Python" likha ho.

[TIP] Hirely Ka Solution:

- **Candidate ke liye:** Ek dashboard jahan wo resume upload karke check kar sakta hai ki job ke sath uska match score kitna hai, kaunse skills missing hain, aur unhe resume mein kaise improve karein.
- **Recruiter ke liye:** Ek system jo automatically applicants ko rank karta hai (exact skills + semantic similarity dono dekh kar), aur ek **RAG Assistant** deta hai jisse recruiter resume ke bare mein sawal pooch sakta hai aur direct evidence dekh sakta hai.

3. User Is Project Mein Kya Karta Hai? (User Journey)

* Candidate Ka Flow:

1. **Signup / Login:** Candidate apna account banata hai (Argon2 password hash hota hai, JWT token milta hai).
2. **Profile & Resume:** Apni details bharta hai aur PDF resume upload karta hai.
3. **Automatic Parsing:** Backend **Docling** use karke resume ka text extract karta hai aur skills ko candidate profile mein save kar deta hai.
4. **Job Search:** Available jobs dekhta hai.

5. **ATS Analysis:** Apply karne se pehle job ke sath apna resume check karta hai (use 0 se 100 ka score aur improvement suggestions milte hain).
6. **Apply:** Ek click mein apply karta hai (database check karta hai ki duplicate application na ho).
7. **Status Tracking:** Dekh sakta hai ki application `applied`, `shortlisted`, `interview`, ya `hired` hui hai.

* Recruiter Ka Flow:

1. **Recruiter Setup:** Login karke apni Company register karta hai.
2. **Job Creation:** Job post karta hai jisme required skills (jaise Python, SQL) aur preferred skills (jaise Docker, Redis) define karta hai.
3. **AI Job Audit:** Publish karne se pehle AI se check karwata hai ki job description clear hai ya nahi.
4. **Applicant Review:** Job ke dashboard par applicants ki ranked list dekhta hai.
5. **AI Evidence Assistant:** Recruiter candidate ke resume ko line-by-line padhne ke bajaye AI se poochta hai: *"Does this candidate have experience building microservices?"* AI resume se extract karke exact quote ke sath answer deta hai.
6. **Status Update:** Candidate ko shortlist ya reject karta hai.

---

4. Backend Request Kaise Process Karta Hai? (Request Lifecycle)

Jab frontend se koi button click hota hai (e.g. "Analyze ATS"), to backend ke andar kya hota hai?

```
[Browser / Frontend]
        |  POST /candidate/ats-analysis (Header: Authorization: Bearer <JWT>, X-Request-ID)
        v
[CorrelationIdMiddleware]
        |  Incoming request ko X-Request-ID deta hai (taaki logs trace ho sakein).
        v
[CORSMiddleware]
        |  Check karta hai ki request allowed frontend URL se aa rahi hai ya nahi.
        v
[RateLimiter Dependency]
        |  Check karta hai: Kya is user ne 1 minute mein allowed limits se zyada AI calls to nahi kiye?
        v
[Authentication Dependency (require_candidate)]
        |  JWT token verify karta hai. Role check karta hai (Recruiter ko candidate endpoint par 403 Forbidden deta hai).
        v
[Pydantic Request Validation]
        |  Check karta hai payload format sahi hai ya nahi (e.g. valid job_id UUID).
        v
[Domain Service (ats_analyzer_service)]
        |  PostgreSQL se Job aur Candidate ka active Resume fetch karta hai.
        |  Deterministic backend calculation: Required Skills Match (45%) + Preferred (15%) + Completeness (10%).
        |  Gemini Embeddings call karta hai: Resume text aur Job text ka Cosine Similarity nikalta hai (30%).
        |  Gemini LLM call karta hai: Qualitative feedback (strengths/weaknesses) generate karta hai.
        v
[Pydantic Response Validation]
        |  Response schema validate hota hai.
        v
[Frontend Response] (HTTP 200 OK with JSON data)
```

---

5. AI Features Kaise Kaam Karte Hain? (AI Mechanics)

A. Resume Parser (Docling)

- **Problem:** Normal PDF parsers (like PyPDF2) multi-column resumes mein text ko mix kar dete hain (left column aur right column ek hi line mein jud jate hain).
- **Hirely Ka Approach:** Hum **Docling** use karte hain. Docling C++ aur deep learning layout detection use karta hai. Wo identify karta hai ki header kahan hai, work experience kahan hai, aur skills kahan hain, phir structured JSON banata hai.

B. Hybrid ATS Scoring Engine

- **Formula Samajho:**

`Final Score = (Required Skills x 0.45) + (Preferred Skills x 0.15) + (Semantic Match x 0.30) + (Resume Completeness x 0.10)`

- **Kyun Kiya Aisa?** Agar hum sirf LLM se bolte: *"Read this resume and give it a score from 1 to 100"*, to LLM Monday ko 80 deta aur Tuesday ko 65 deta! Humne numerical scoring ko purely deterministic math banaya, aur Gemini ko sirf explanation likhne ke liye use kiya.

C. Semantic Matching (Embeddings)

- **Concept:** Agar job description mein likha hai *"Experience with relational databases"* aur candidate ke resume mein likha hai *"PostgreSQL, MySQL query optimization"*.
- Exact keyword matcher bolega: *"0% match! 'Relational databases' word nahi mila."*

- Lekin **Gemini Embeddings** dono sentences ko dense vectors (768 numbers ki list) mein convert karta hai. Phir hum math ka **Cosine Similarity** calculate karte hain. Embeddings samajh jate hain ki PostgreSQL hi relational database hai, aur candidate ko high similarity score milta hai!

D. AI Cover Letter Generator

- Model ko sirf candidate ka actual parsed resume experience aur target job description di jati hai.
- Prompt mein strict rule hota hai: *"Do not invent any company, date, or metric not present in the candidate context."* Isse fake baatein nahi banti.

---

6. Database Ka Role Kya Hai? (PostgreSQL & SQLAlchemy)

Hirely **PostgreSQL 15** use karta hai kyunki recruitment data highly relational hota hai:

- `users` table mein login credentials aur role hota hai (`candidate` ya `recruiter`).
- `candidates` aur `recruiters` users se `1:1` link hote hain.
- `companies` table recruiter se linked hoti hai.
- `jobs` table recruiter se linked hoti hai.
- `applications` table candidate aur job ke beech ka bridge hai. Database level par **Unique Constraint** laga hai: `UniqueConstraint('candidate_id', 'job_id')`. Iska matlab ek candidate ek job par do baar apply nahi kar sakta (backend crash hone ke bajaye clean HTTP 409 Conflict return karta hai).
- **SQLAlchemy 2.0:** Code mein raw SQL strings likhne ke bajaye Python classes (ORM models) use hoti hain.
- **Alembic:** Database migrations manage karta hai. Jab bhi koi naya column banta hai, Alembic script chalti hai taaki production data bina delete hue schema update ho sake.

---

7. RAG Kya Karta Hai Aur Kyun Zaroori Hai? (Candidate Evidence Assistant)

RAG Ka Full Form:

Retrieval-Augmented Generation (Pehle dhoondho, phir answer generate karo).

Real-World Example:

Maan lo candidate ka resume 4 pages ka hai. Recruiter poochta hai:

"Has this candidate ever handled production Kubernetes deployment issues?"

Naive LLM Ka Khatra:

Agar tum poora resume seedhe LLM ko bhejte ho bina grounding ke, to LLM kai baar hallucinate karke bol deta hai: *"Yes, candidate is an expert in Kubernetes"*, chahe usne sirf ek blog padha ho.

Hirely Ka RAG Pipeline:

1. **Chunking:** Resume ko meaningful paragraphs (chunks) mein toda jata hai.
2. **Retrieval:** Recruiter ke sawal ka vector embedding banakar resume chunks ke sath compare kiya jata hai. Sabse relevant 3-5 chunks nikale jate hain.
3. **Grounded Generation:** Gemini ko prompt bheja jata hai:

"Answer this question ONLY using the provided resume chunks. You MUST provide the exact verbatim quote from the resume that proves your answer."

4. **Backend Substring Verification (Masterstroke):** Jab Gemini answer aur quote return karta hai, hamara backend code Python mein verify karta hai:

```
assert quote in retrieved_chunk_text
```

Agar model ne fake quote banaya, to backend us citation ko invalid mark kar deta hai! Isse recruiter ko 100% verified evidence milta hai.

---

8. Docker Kyun Use Kiya? (Containerization Explained)

"Mere Machine Pe To Chal Raha Tha!" Problem:

Aksar software developer ke laptop par chalta hai, lekin doosre computer par Python version, PostgreSQL version, ya Node version alag hone par crash ho jata hai.

Docker Solution:

Docker poore system ko 4 isolated containers mein pack kar deta hai jo `docker compose up --build -d` chalate hi start ho jate hain:

1. `hirely-postgres`: PostgreSQL 15 database container. Data `postgres_data` volume mein save rehta hai, container restart hone par bhi data delete nahi hota.

2. **`hirely-migration`:** Ek temporary container jo database ready hone ke baad start hota hai, **`alembic upgrade head`** chalata hai, aur tables banne ke baad band ho jata hai.
3. **`hirely-backend`:** FastAPI application container. Ye migration complete hone ke baad start hota hai taaki kabhi "table doesn't exist" error na aaye.
4. **`hirely-frontend`:** React app ko production bundle mein build karke lightweight Nginx server par host karta hai (port 3000 par).

---

9. Project Mein Kya Limitations Hain? (Real Production Gaps)

Interview mein jab interviewer pooche: *"Is project ki limitations kya hain?"*, to mature developer ki tarah honest answers dene chahiye:

1. **In-Memory Rate Limiting:** Abhi rate limiting Python process ki memory mein hoti hai. Agar hum multi-worker cluster banayein, to memory share nahi hogi. Production mein **Redis** lagana padega.
2. **Synchronous Docling Parsing:** Abhi PDF parsing synchronous hoti hai (request aati hai, parser chalta hai, phir response jata hai). Agar 100 log ek sath 10 MB ki PDF upload karein, to CPU block ho sakta hai. Production mein **Celery / RabbitMQ** background worker queue chahiye.
3. **In-Memory Vector Search:** Candidate matching ke liye vector cosine similarity memory mein calculate hoti hai. Agar 100,000 candidates honge, to database mein **pgvector** extension use karke indexing karni padegi.
4. **Docker Health $\neq$ Production Ready:** Docker compose local development ke liye best hai. Production ke liye managed database (AWS RDS), SSL certificates (HTTPS), aur managed object storage chahiye.

---

10. Quick Revision Cheat-Sheet

Sawal (Concept)	1-Line Hinglish Answer
Hirely kya hai?	Ek dual-sided AI recruitment platform jo candidates ko ATS fit aur recruiters ko grounded candidate evidence deta hai.
FastAPI kyun use kiya?	High performance async capabilities aur native Pydantic v2 data validation ke liye.
ATS score kaise banta hai?	45% Required Skills + 15% Preferred Skills + 30% Semantic Embeddings + 10% Completeness.
Docling kya hai?	Ek deep-learning based document layout parser jo complex PDF resumes ko accurately structured text mein convert karta hai.
RAG kaise hallucination rokta hai?	Context ko resume chunks tak limit karke, model se exact quote mangta hai aur backend code us quote ko source text se verify karta hai.
Argon2 kya hai?	Password hashing ka gold standard algorithm jo brute force aur GPU attacks ke khilaf memory-hard security deta hai.
Cloudflare R2 kyun lagaya?	Stateless cloud hosting (jaise Render) par local disk wipe ho jati hai; R2 S3-compatible zero-egress object storage deta hai.